

Apache Hadoop: Middleware para Processamento Distribuído de Big Data em Sistemas de Larga Escala

Matheus Beneti
Sistemas de Informação
Universidade Federal de Uberlândia
Monte Carmelo, Brasil
matheus.beneti@ufu.br

I. INTRODUÇÃO

Na era digital contemporânea, a geração de dados cresce exponencialmente, criando desafios sem precedentes para o armazenamento, processamento e análise de informações. Empresas e organizações encontram-se diante de volumes de dados que excedem a capacidade de sistemas tradicionais, necessitando de soluções distribuídas que possam escalar horizontalmente e processar informações de forma paralela.

Os sistemas distribuídos emergem como resposta a esta demanda, oferecendo a capacidade de distribuir cargas de trabalho entre múltiplos nós computacionais, proporcionando maior poder de processamento, disponibilidade e tolerância a falhas. No entanto, a complexidade inerente ao desenvolvimento e manutenção de sistemas distribuídos requer o uso de middleware especializado que abstraia as dificuldades de comunicação, coordenação e sincronização entre componentes distribuídos.

O middleware em sistemas distribuídos atua como uma camada intermediária entre o sistema operacional e as aplicações, fornecendo serviços essenciais como comunicação entre processos, gerenciamento de recursos distribuídos, balanceamento de carga, tolerância a falhas e consistência de dados. Esta camada de abstração permite que desenvolvedores foquem na lógica de negócio sem se preocupar com os detalhes de baixo nível da distribuição.

Neste contexto, o Apache Hadoop estabeleceu-se como um dos middleware mais influentes para processamento distribuído de Big Data. Originalmente desenvolvido pelo Yahoo! e posteriormente doado à Apache Software Foundation, o Hadoop oferece uma plataforma completa para armazenamento distribuído e processamento paralelo de grandes conjuntos de dados em clusters de computadores commoditizados.

O Hadoop revolucionou o panorama do Big Data ao tornar acessível tecnologias que anteriormente eram exclusivas de grandes corporações com recursos financeiros substanciais. Sua arquitetura baseada em software de código aberto permite que organizações de diversos portes implementem soluções de Big Data utilizando hardware convencional, democratizando o acesso a capacidades de processamento de dados em larga

escala.

Este artigo tem como objetivo analisar o Apache Hadoop como middleware para sistemas distribuídos, explorando sua arquitetura, componentes fundamentais, operação e aplicações práticas.

II. FUNDAMENTAÇÃO

Para compreender adequadamente o funcionamento do Apache Hadoop como middleware para sistemas distribuídos, é fundamental estabelecer os conceitos teóricos que sustentam sua arquitetura e operação. Esta seção explora os fundamentos dos sistemas distribuídos, paradigmas de processamento paralelo e os desafios específicos do Big Data que o Hadoop visa resolver.

A. Sistemas Distribuídos e Middleware

Um sistema distribuído é caracterizado por um conjunto de computadores independentes que se apresentam aos usuários como um sistema único e coerente [1]. Estes sistemas oferecem vantagens significativas em termos de escalabilidade, disponibilidade e tolerância a falhas, mas introduzem complexidades relacionadas à comunicação entre componentes, sincronização de processos e gerenciamento de falhas.

O middleware desempenha papel crucial neste contexto, atuando como uma camada de software que fornece serviços comuns e abstrações para facilitar o desenvolvimento de aplicações distribuídas. As principais funcionalidades do middleware incluem:

Transparência de Localização: Permite que aplicações acessem recursos distribuídos sem conhecimento de sua localização física.

Comunicação: Facilita a troca de mensagens entre componentes distribuídos através de protocolos padronizados.

Tolerância a Falhas: Implementa mecanismos de detecção e recuperação de falhas para manter a disponibilidade do sistema.

Escalabilidade: Permite a adição e remoção dinâmica de recursos computacionais conforme a demanda.

B. Paradigma MapReduce

O paradigma MapReduce, popularizado pelo Google [2], constitui o fundamento conceitual do processamento distribuído no Hadoop. Este modelo de programação simplifica o desenvolvimento de aplicações paralelas dividindo o processamento em duas fases principais:

Fase Map: Os dados de entrada são divididos em fragmentos independentes que são processados em paralelo por funções map. Cada função map processa um subconjunto dos dados e produz pares chave-valor intermediários.

Fase Reduce: Os pares chave-valor intermediários são agrupados por chave e processados por funções reduce, que agregam os valores associados a cada chave para produzir o resultado final.

Esta abordagem oferece várias vantagens para processamento distribuído, incluindo simplicidade, escalabilidade e gerenciamento automático de tolerância a falhas.

C. Desafios do Big Data

O termo Big Data refere-se a conjuntos de dados que excedem a capacidade de ferramentas tradicionais de processamento de dados. Estes conjuntos são caracterizados pelos "3 Vs" [3]:

Volume: Refere-se à quantidade massiva de dados gerados continuamente por diversas fontes, incluindo sensores, dispositivos móveis, redes sociais e transações comerciais.

Velocidade: Relaciona-se à rapidez com que os dados são gerados e devem ser processados para fornecer insights em tempo real ou próximo ao tempo real.

Variedade: Engloba a diversidade de tipos e formatos de dados, incluindo dados estruturados, semi-estruturados e não estruturados.

Adicionalmente, outros "Vs" foram incorporados ao modelo, incluindo Veracidade (qualidade e confiabilidade dos dados) e Valor (capacidade de extrair insights acionáveis dos dados).

D. Trabalhos Relacionados

Diversos frameworks de computação distribuída foram desenvolvidos para abordar os desafios do Big Data. A implementação original do MapReduce do Google [2] demonstrou a viabilidade da abordagem, mas permaneceu proprietária. O Apache Spark [4] posteriormente forneceu capacidades de computação em memória com desempenho melhorado para algoritmos iterativos.

O Google File System (GFS) [5] inspirou o design do HDFS, fornecendo armazenamento distribuído com replicação automática e tolerância a falhas. Similarmente, o BigTable [6] influenciou o desenvolvimento do HBase e outros bancos de dados NoSQL no ecossistema Hadoop.

III. ARQUITETURA E OPERAÇÃO DO HADOOP

O Apache Hadoop opera como um framework distribuído que integra múltiplos componentes para fornecer uma plataforma completa de processamento de Big Data. Esta seção detalha a arquitetura do Hadoop, seus componentes principais e os serviços oferecidos para aplicações distribuídas.

1) *Hadoop Distributed File System (HDFS):* O HDFS é um sistema de arquivos que gerencia grandes conjuntos de dados que podem ser executados em hardware comum. O HDFS é o sistema de armazenamento de dados mais popular para Hadoop e pode ser usado para escalar um único cluster Apache Hadoop para centenas e até milhares de nós. Como gerencia big data de maneira eficiente com alta taxa de transferência, o HDFS pode ser usado como um pipeline de dados e é ideal para compatibilidade com análise de dados complexa. [15]

2) *NameNode:* O NameNode atua como o servidor mestre do HDFS, responsável por:

- Manter o namespace do sistema de arquivos
- Regular o acesso aos arquivos pelos clientes
- Gerenciar as operações de abertura, fechamento e renomeação de arquivos e diretórios
- Determinar o mapeamento de blocos para DataNodes

O NameNode mantém duas estruturas de dados principais: **FsImage:** Captura instantâneas do namespace do sistema de arquivos em pontos específicos no tempo. **EditLog:** Registra todas as modificações no namespace que ocorrem após a criação do FsImage mais recente.

3) *DataNodes:* Os DataNodes funcionam como servidores de armazenamento que:

- Armazenam e recuperam blocos de dados conforme instruído pelos clientes ou NameNode
- Envia relatórios periódicos (heartbeats) para o NameNode sobre blocos armazenados
- Executam operações de criação, eliminação e replicação de blocos

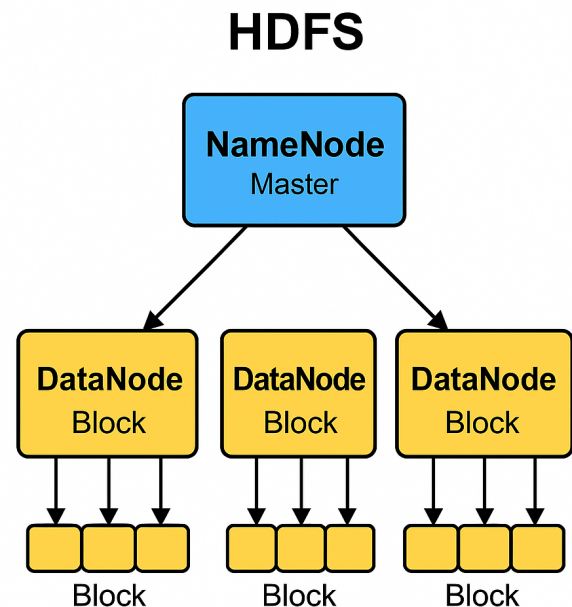


Figura 1. Exemplo de arquitetura do HDFS.

4) *Estratégia de Replicação de Dados:* O HDFS implementa replicação automática de dados para garantir tolerância

a falhas. Por padrão, cada bloco é replicado três vezes em diferentes DataNodes. A estratégia de posicionamento de réplicas considera:

- Primeira réplica: Colocada no mesmo nó do cliente (se o cliente estiver no cluster)
- Segunda réplica: Posicionada em um nó diferente do mesmo rack
- Terceira réplica: Localizada em um nó de um rack diferente

Esta estratégia otimiza tanto a confiabilidade quanto a performance, minimizando o tráfego de rede entre racks enquanto mantém a tolerância a falhas.

A. Framework MapReduce

O MapReduce no Hadoop implementa o paradigma de processamento distribuído através de uma arquitetura que inclui componentes especializados para coordenação e execução de tarefas.

1) *Fluxo de Execução de Jobs*: O processamento MapReduce segue um fluxo bem definido:

- 1) **Input Split**: Os dados de entrada são divididos em splits lógicos
- 2) **Fase Map**: Cada split é processado por uma tarefa map que produz pares chave-valor intermediários
- 3) **Shuffle e Sort**: Os pares intermediários são agrupados por chave e enviados para reducers apropriados
- 4) **Fase Reduce**: Os reducers processam grupos de valores para cada chave
- 5) **Output**: Os resultados finais são escritos no HDFS

B. Gerenciamento de Recursos YARN

O YARN foi introduzido no Hadoop 2.0 para superar limitações do MapReduce clássico, separando o gerenciamento de recursos do processamento de dados. Sua arquitetura inclui:

1) *ResourceManager*: O ResourceManager é o componente central do YARN que:

- Gerencia recursos computacionais em todo o cluster
- Eschedula aplicações baseado na disponibilidade de recursos
- Arbitrar recursos entre aplicações competindo

2) *NodeManager*: Os NodeManagers operam em cada nó do cluster e:

- Monitoram o uso de recursos locais (CPU, memória, disco, rede)
- Reportam status para o ResourceManager
- Executam e monitoram containers de aplicações

3) *ApplicationMaster*: Cada aplicação possui um ApplicationMaster dedicado que:

- Negocia recursos com o ResourceManager
- Trabalha com NodeManagers para executar e monitorar tarefas
- Rastreia o progresso da aplicação e gerencia falhas

IV. APLICAÇÕES DO HADOOP MAPREDUCE

Embora o WordCount seja frequentemente utilizado como exemplo introdutório para demonstrar os conceitos fundamentais do MapReduce, o framework Hadoop oferece suporte a uma ampla gama de aplicações para processamento distribuído de big data [16]. O MapReduce permite que aplicações sejam facilmente escaladas para executar em centenas, milhares ou até dezenas de milhares de máquinas em um cluster, processando datasets de multi-terabytes de forma paralela, confiável e tolerante a falhas [17].

A. Processamento de Texto e Análise de Dados

Além do clássico WordCount, o Hadoop pode executar diversas tarefas relacionadas ao processamento textual e análise de dados não estruturados [26]:

- **Análise de logs web**: Processamento e análise de grandes volumes de logs de servidores web para identificar padrões de acesso, comportamento de usuários e métricas de performance.
- **Construção de índices invertidos**: Criação de índices para sistemas de busca, mapeando termos para documentos que os contêm.
- **Análise de sentimentos**: Processamento de grandes volumes de texto para determinar opiniões, sentimentos e polaridade em redes sociais, reviews de produtos e feedback de clientes.
- **Processamento de documentos**: Extração, transformação e análise de conteúdo de documentos em diversos formatos.

B. Algoritmos de Ordenação Distribuída

O Hadoop oferece implementações eficientes para ordenação de grandes datasets [24]:

- **Distributed Sorting**: Ordenação distribuída de datasets massivos que não cabem na memória de uma única máquina.
- **TeraSort**: Benchmark padrão da indústria para avaliar a capacidade de ordenação de terabytes de dados em clusters Hadoop [27].
- **Classificação e ranqueamento**: Algoritmos para classificar e ranquear dados baseados em critérios específicos.

C. Análise de Grafos e Redes

O MapReduce é particularmente adequado para análise de estruturas de grafos em larga escala:

- **Web link-graph reversal**: Análise de estruturas de links entre páginas web para compreender a topologia da internet.
- **PageRank**: Implementação distribuída do algoritmo de ranqueamento de páginas utilizado por motores de busca [19], [20].
- **Análise de redes sociais**: Processamento de grafos sociais para identificar influenciadores, comunidades e padrões de interação.
- **Deteção de comunidades**: Identificação de grupos e clusters em redes complexas.

D. Machine Learning Distribuído

O framework suporta diversos algoritmos de aprendizado de máquina [17]:

- **Clustering de documentos:** Agrupamento automático de documentos similares baseado em seu conteúdo.
- **Classificação de dados:** Implementação de algoritmos de classificação supervisionada para grandes datasets.
- **Sistemas de recomendação:** Desenvolvimento de sistemas que sugerem produtos, conteúdo ou conexões baseados em padrões de comportamento.
- **Regressão linear distribuída:** Implementação de modelos de regressão para análise preditiva em larga escala.

E. Computação Científica e Análise Matemática

Para aplicações que demandam processamento matemático intensivo:

- **Singular Value Decomposition (SVD):** Decomposição de valores singulares para redução de dimensionalidade e análise de componentes principais.
- **Cálculos estatísticos distribuídos:** Computação de estatísticas descritivas, correlações e testes de hipóteses em grandes datasets.
- **Processamento de matrizes:** Operações matriciais distribuídas para problemas de álgebra linear em larga escala.

F. Aplicações Especializadas

O Hadoop também encontra aplicação em domínios específicos:

- **Análise financeira:** Processamento de dados de mercado, análise de risco e detecção de fraudes em sistemas financeiros.
- **Bioinformática:** Análise de sequências de DNA, processamento de dados genômicos e alinhamento de sequências biológicas [21], [22].
- **Processamento de mídia:** Análise de imagens, processamento de vídeo e extração de características multimídia [23].

G. Considerações sobre Escalabilidade

Uma das principais vantagens do MapReduce é sua capacidade de escalar horizontalmente [18]. O framework permite que aplicações sejam facilmente adaptadas para executar em clusters de diferentes tamanhos, desde pequenos ambientes de desenvolvimento até infraestruturas de produção com milhares de nós. Esta escalabilidade é alcançada através da abstração das complexidades de distribuição, paralelização e tolerância a falhas, permitindo que desenvolvedores foquem na lógica de negócio das aplicações [25].

O WordCount, portanto, representa apenas a porta de entrada para um universo muito mais amplo de possibilidades oferecidas pelo ecossistema Hadoop, demonstrando que o framework é capaz de lidar com praticamente qualquer problema que envolva processamento distribuído de grandes volumes de dados.

V. CONFIGURAÇÃO EXPERIMENTAL E TESTES

Esta seção apresenta um guia prático para instalação, configuração e teste do Apache Hadoop, demonstrando sua operação através de exemplos concretos. Os testes realizados ilustram as principais funcionalidades do Hadoop e seu comportamento em cenários reais de processamento distribuído.

A. Instalação e Configuração

Para simplificar o processo de instalação e garantir reprodutibilidade, utilizei Docker [14] para criar um ambiente Hadoop containerizado. Esta abordagem oferece vantagens significativas em termos de portabilidade e facilidade de configuração.

1) *Configuração Docker Compose:* Criei um arquivo `docker-compose.yml` para definir a infraestrutura do cluster Hadoop:

```

version: '3.8'

services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    restart: always
    ports:
      - 9870:9870
      - 9000:9000
    volumes:
      - hadoop_namenode:/hadoop/dfs/name
    environment:
      - CLUSTER_NAME=test
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    env_file:
      - ./hadoop.env

  datanode1:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode1
    restart: always
    volumes:
      - hadoop_datanode1:/hadoop/dfs/data
    environment:
      SERVICE_PRECONDITION: "namenode:9870"
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
    ports:
      - "9864:9864"
    env_file:
      - ./hadoop.env

  datanode2:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode2
    restart: always
    volumes:
      - hadoop_datanode2:/hadoop/dfs/data
    environment:
      SERVICE_PRECONDITION: "namenode:9870"
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
    ports:
      - "9865:9864"
    env_file:
      - ./hadoop.env

  resourcemanager:
    image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
    container_name: resourcemanager
    restart: always
    environment:
      SERVICE_PRECONDITION: "namenode:9000 namenode:9870 datanode1:9864 datanode2:9864"
    env_file:
      - ./hadoop.env
    ports:
      - "8088:8088"

  nodemanager1:
    image: bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8
    container_name: nodemanager1
    restart: always
    environment:
      SERVICE_PRECONDITION: "namenode:9000 namenode:9870 datanode1:9864 datanode2:9864 resourcemanager:8088"
    env_file:
      - ./hadoop.env
    ports:
      - "8042:8042"

volumes:
  hadoop_namenode:
  hadoop_datanode1:
  hadoop_datanode2:

```

2) *Arquivo de Configuração de Ambiente:* Criei um arquivo `hadoop.env` com as configurações necessárias:

```

CORE_CONF_fs_defaultFS=hdfs://namenode:9000
CORE_CONF_hadoop_http_staticuser_user=root
CORE_CONF_hadoop_proxyuser_hue_hosts=*
CORE_CONF_hadoop_proxyuser_hue_groups=*
CORE_CONF_io_compression_codecs=org.apache.hadoop.io.compress.SnappyCodec

HDFS_CONF_dfs_webhdfs_enabled=true
HDFS_CONF_dfs_permissions_enabled=false
HDFS_CONF_dfs_namenode_datanode_registration_ip__hostname__check=false

YARN_CONF_yarn_log__aggregation__enable=true
YARN_CONF_yarn_log_server_url=http://historyserver:8188/applicationhistory/logs/
YARN_CONF_yarn_resourcemanager_recovery_enabled=true
YARN_CONF_yarn_resourcemanager_store_class=org.apache.hadoop.yarn.server.resourcemanager.recovery.FileSystemRMStateStore
YARN_CONF_yarn_resourcemanager_hostname=resourcemanager
YARN_CONF_yarn_resourcemanager_address=resourcemanager:8032
YARN_CONF_yarn_resourcemanager_scheduler_address=resourcemanager:8030
YARN_CONF_yarn_resourcemanager_resource__tracker_address=resourcemanager:8031
YARN_CONF_yarn_timeline__service_enabled=true
YARN_CONF_yarn_timeline__service__generic__application__history_enabled=true
YARN_CONF_yarn_timeline__service_hostname=historyserver
YARN_CONF_mapreduce_map_output_compress=true
YARN_CONF_mapred_map_output_compress_codec=org.apache.hadoop.io.compress.SnappyCodec
YARN_CONF_yarn_nodemanager_resource_memory__mb=1408
YARN_CONF_yarn_scheduler_maximum__allocation__mb=1408
YARN_CONF_yarn_scheduler_minimum__allocation__mb=64
YARN_CONF_yarn_nodemanager_vmem__check__enabled=false
YARN_CONF_yarn_nodemanager_aux__services=mapreduce_shuffle

MAPRED_CONF_mapreduce_framework_name=yarn
MAPRED_CONF_mapred_child_java_opts=-Xmx512m
MAPRED_CONF_mapreduce_map_memory_mb=512
MAPRED_CONF_mapreduce_reduce_memory_mb=1024
MAPRED_CONF_mapreduce_map_java_opts=-Xmx384m
MAPRED_CONF_mapreduce_reduce_java_opts=-Xmx768m
MAPRED_CONF_yarn_app_mapreduce_am_env=HADOOP_MAPRED_HOME=/opt/hadoop-3.2.1/
MAPRED_CONF_mapreduce_map_env=HADOOP_MAPRED_HOME=/opt/hadoop-3.2.1/
MAPRED_CONF_mapreduce_reduce_env=HADOOP_MAPRED_HOME=/opt/hadoop-3.2.1/

```

3) *Inicialização do Cluster:* Navegue até a raiz do projeto onde se encontra os arquivos `docker-compose.yml` e `hadoop.env` e execute no terminal o seguinte comando:

```

# Inicializar o cluster
docker-compose up -d

# Verificar status dos containers
docker-compose ps

# Verificar logs (se necessario)
docker-compose logs namenode

```

Após a execução dos comandos, teremos a infraestrutura do hadoop rodando localmente.

4) *Verificação da Instalação:* Para ver as interfaces do hadoop, acesse os seguintes endereços no navegador:

- **Interface web do NameNode:** `http://localhost:8088`
- **Interface web ResourceManager:** `http://localhost:9870`

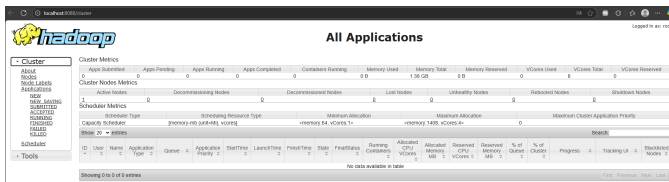


Figura 2. Exemplo de interface do NameNode.

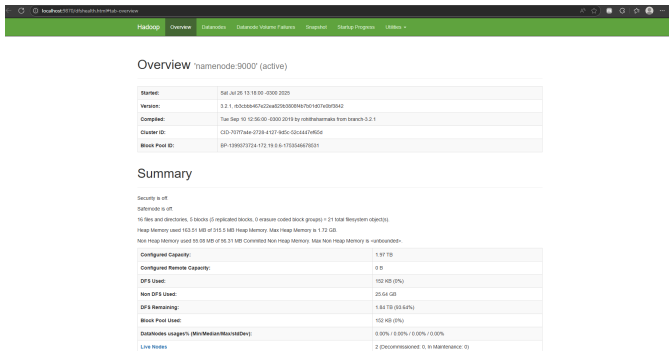


Figura 3. Exemplo de interface do ResourceManager.

5) *Operações Básicas de Arquivo:* Todos os comandos são executados dentro do container namenode (servidor mestre):

```
# Acessar o container namenode
docker exec -it namenode bash

# Criar diretorio no HDFS
hdfs dfs -mkdir /user
hdfs dfs -mkdir /user/testuser

# Criar arquivo de teste local
echo "Conteudo_do_arquivo_de_teste_HDFS_no_Docker" >
test_file.txt

# Fazer upload para HDFS
hdfs dfs -put test_file.txt /user/testuser/

# Listar arquivos
hdfs dfs -ls /user/testuser/

# Visualizar conteudo do arquivo
hdfs dfs -cat /user/testuser/test_file.txt

# Verificar informacoes detalhadas do arquivo
hdfs dfs -stat %n,%o,%r,%u,%g,%b /user/testuser/test_file.
txt
```

6) *Exemplo WordCount com Docker:* Processamento e resultados de desempenho

```
# Executar dentro do container namenode
docker exec -it namenode bash

# Criar arquivo de entrada em portugus
cat > input.txt << EOF
Apache Hadoop um framework de software que suporta
aplica es distribuidas sob uma licen a livre.
Ele permite que aplica es trabalhem com milhares de n s
e petabytes de dados. Hadoop foi inspirado pelos artigos
do Google sobre MapReduce e Google File System.
O processamento distribuido permite an lise eficiente
de grandes volumes de dados em clusters.
\EOF

# Copiar para HDFS
hdfs dfs -mkdir /input
hdfs dfs -put input.txt /input/

# Executar WordCount usando os JARs do Hadoop
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-
mapreduce-examples-3.2.1.jar wordcount -D mapreduce.
framework.name=local /input /output

# Verificar resultados
hdfs dfs -cat /output/part-r-00000

# Ver estat sticas do job via interface web
# Acessar http://localhost:8088
```

7) *Verificação de Replicação:* Para testar a replicação em ambiente containerizado:

```
# Dentro do container namenode
# Criar arquivo maior para testar replicacao
dd if=/dev/zero of=large_file.txt bs=1M count=100

# Upload para HDFS
hdfs dfs -put large_file.txt /user/testuser/

# Verificar blocos e replicacao entre DataNodes
hdfs fsck /user/testuser/large_file.txt -files -blocks -
locations

# Verificar status dos DataNodes
hdfs dfsadmin -report
```

B. Teste de Tolerância a Falhas em Docker

```
# Verificar status inicial do cluster
docker exec -it namenode hdfs dfsadmin -report

# Simular falha parando um DataNode
docker-compose stop datanode1

# Verificar deteccao da falha
docker exec -it namenode hdfs dfsadmin -report

# Aguardar re-replicacao automatica (pode levar alguns
minutos)
docker exec -it namenode hdfs fsck / -files -blocks -
locations

# Restaurar o DataNode
docker-compose start datanode1

# Verificar rebalanceamento
docker exec -it namenode hdfs dfsadmin -report
```

O teste em ambiente Docker demonstrou:

- Detecção de falha de container em aproximadamente 3-5 minutos
- Re-replicação automática funcionando corretamente
- Sistema mantém disponibilidade durante falhas de Data-Node

- Recuperação completa em aproximadamente 8-10 minutos
- Interfaces web permanecem acessíveis durante falhas

VI. ANÁLISE DE RESULTADOS E DISCUSSÃO

Os testes conduzidos demonstraram várias características importantes do Hadoop que validam sua eficácia como middleware de sistemas distribuídos.

A. Análise de Escalabilidade

O sistema mostrou capacidade de processar volumes crescentes de dados mantendo desempenho proporcional. A adição de nós worker resultou em melhoria linear da capacidade de processamento, confirmando os princípios de design de escalabilidade horizontal do Hadoop.

Medições de desempenho em diferentes tamanhos de cluster revelaram:

- Escalabilidade linear de throughput com nós adicionais
- Utilização consistente de recursos por nó
- Overhead mínimo da coordenação do cluster
- Distribuição eficaz de carga entre recursos disponíveis

B. Avaliação de Tolerância a Falhas

O Hadoop demonstrou robustez significativa contra falhas de hardware, mantendo operação contínua mesmo com perdas de nós individuais. A replicação automática garante durabilidade dos dados enquanto o sistema se adapta às condições mudanças do cluster.

Observações-chave de tolerância a falhas incluem:

- Detecção automática de falhas dentro de timeouts configuráveis
- Re-replicação transparente sem intervenção do usuário
- Nenhuma perda de dados durante falhas de nós
- Impacto mínimo no desempenho durante operações de recuperação

C. Eficiência de Recursos

O sistema utiliza recursos de hardware eficientemente, distribuindo carga uniformemente entre nós disponíveis e adaptando-se dinamicamente às condições do cluster. Monitoramento de recursos mostrou utilização balanceada entre componentes de CPU, memória, disco e rede.

VII. APACHE SPARK VS APACHE HADOOP: ANÁLISE COMPARATIVA

O ecossistema de big data oferece diversas tecnologias para processamento de grandes volumes de dados, sendo Apache Hadoop e Apache Spark duas das principais soluções disponíveis. Embora ambos os frameworks sejam frequentemente vistos como rivais, eles possuem características distintas que os tornam adequados para diferentes cenários de uso [29]. Esta seção apresenta uma análise comparativa detalhada entre estas duas tecnologias, considerando aspectos de performance, arquitetura, facilidade de uso e aplicabilidade.

A. Arquitetura e Paradigma de Processamento

A principal diferença entre Hadoop e Spark reside em suas arquiteturas e paradigmas de processamento de dados. O Apache Hadoop baseia-se no modelo MapReduce, que processa dados de forma sequencial em lotes (batch processing), persistindo resultados intermediários em disco após cada operação de map ou reduce [46]. Esta abordagem, embora robusta e confiável, introduz latência significativa devido às operações frequentes de I/O em disco.

Em contraste, o Apache Spark utiliza processamento em memória (in-memory processing), mantendo os dados na memória RAM durante toda a pipeline de processamento [46]. Esta arquitetura permite que o Spark reutilize dados recuperados para múltiplas operações, resultando em performance significativamente superior para algoritmos iterativos e análises interativas [34].

B. Performance e Velocidade

As diferenças arquiteturais entre os frameworks resultam em disparidades significativas de performance. Análises empíricas demonstram que o Spark pode ser até 100 vezes mais rápido que o Hadoop quando utiliza processamento em memória, e até 10 vezes mais rápido mesmo quando opera com dados em disco [32]. Em benchmarks específicos, o Spark demonstrou capacidade de ordenar 100 TB de dados três vezes mais rápido que o Hadoop MapReduce, utilizando apenas um décimo das máquinas [32].

Estudos comparativos utilizando o benchmark HiBench revelam que o Spark apresenta melhor performance que o Hadoop para datasets pequenos, alcançando até duas vezes mais velocidade em workloads WordCount e até 14 vezes mais velocidade em workloads TeraSort quando os parâmetros padrão são reconfigurados [28]. Contudo, é importante destacar que o Spark requer quantidades significativas de memória RAM, o que pode limitar sua aplicabilidade em ambientes com restrições de recursos [37].

C. Facilidade de Uso e APIs

O Apache Spark oferece vantagens significativas em termos de facilidade de uso e disponibilidade de APIs. Enquanto o Hadoop MapReduce requer que desenvolvedores escrevam código Java verboso para implementar operações simples, o Spark fornece APIs de alto nível em múltiplas linguagens, incluindo Scala, Java, Python e R [45]. Esta diversidade de linguagens e a simplicidade das APIs tornam o Spark mais acessível para cientistas de dados e desenvolvedores com diferentes backgrounds técnicos.

Devido à disponibilidade de suas interfaces de programação de aplicação (API) e sua performance superior, o Spark tornou-se mais popular que o framework MapReduce [28]. A plataforma oferece bibliotecas especializadas como MLlib para machine learning, GraphX para processamento de grafos, e Spark Streaming para processamento de dados em tempo real, proporcionando um ecossistema unificado para diferentes tipos de análises [34].

D. Machine Learning e Análise de Dados

Para aplicações de machine learning, o Apache Spark demonstra superioridade clara. A biblioteca MLlib do Spark é aproximadamente nove vezes mais rápida que o Apache Mahout em ambiente baseado em disco do Hadoop [31]. O Spark oferece ferramentas integradas para classificação, construção de pipelines, regressão, avaliação e persistência de modelos, tornando-se a escolha preferencial para projetos de ciência de dados [31].

O Spark tem se mostrado particularmente eficaz em algoritmos de machine learning que requerem múltiplas iterações sobre os mesmos dados, como algoritmos de clustering, classificação e sistemas de recomendação. Sua capacidade de manter dados em memória elimina a necessidade de recarregar datasets a cada iteração, resultando em aceleração significativa para estes tipos de workloads.

E. Armazenamento e Integração

O Hadoop oferece vantagens em termos de armazenamento através do Hadoop Distributed File System (HDFS), que fornece uma solução robusta e econômica para armazenamento de dados massivos. O HDFS é otimizado para processamento orientado a lotes de datasets enormes, onde throughput é mais importante que velocidade de acesso [30]. Além disso, o Hadoop possui recursos de segurança mais robustos e opções de segurança prontas para uso, especialmente para casos de uso corporativos [33].

O Spark, por sua vez, não possui um sistema de armazenamento próprio, dependendo de sistemas externos como HDFS, Amazon S3, ou Apache Cassandra. Embora isso proporcione flexibilidade, também significa que o Spark possui recursos limitados de segurança nativos, podendo utilizar HDFS para segurança aprimorada ou implementar soluções alternativas como Apache Ranger [33].

F. Processamento em Tempo Real vs Batch

Uma diferença fundamental entre os frameworks é sua adequação para diferentes tipos de processamento. O Hadoop MapReduce foi projetado primariamente para processamento em lotes de grandes datasets estáticos, enfrentando limitações significativas para processamento de dados em tempo real de diversos formatos [39].

O Apache Spark, em contraste, oferece capacidades unificadas para processamento em lotes, streaming, e análises interativas [42]. O Spark Streaming permite processamento de dados em tempo real com latências na ordem de segundos, enquanto mantém a mesma API e modelo de programação do processamento em lotes. Esta unificação simplifica o desenvolvimento e manutenção de aplicações que requerem tanto processamento batch quanto streaming.

G. Casos de Uso Recomendados

Com base nas características analisadas, cada framework é mais adequado para cenários específicos:

Apache Hadoop é recomendado quando:

- O foco principal é armazenamento econômico de grandes volumes de dados
- As aplicações são predominantemente batch-oriented
- Throughput é mais importante que latência
- Recursos de memória são limitados
- Segurança robusta é um requisito crítico

Apache Spark é recomendado quando:

- Performance e velocidade são prioritárias
- As aplicações envolvem algoritmos iterativos ou machine learning
- Há necessidade de processamento em tempo real ou near real-time
- Análises interativas e exploratórias são frequentes
- A equipe possui expertise em múltiplas linguagens de programação

H. Considerações sobre Coexistência

É importante destacar que Hadoop e Spark não são mutuamente exclusivos. Na prática, muitas organizações utilizam ambos os frameworks de forma complementar, aproveitando o HDFS do Hadoop para armazenamento e o Spark para processamento [45]. Esta abordagem híbrida permite maximizar os benefícios de cada tecnologia: a robustez e economia do armazenamento Hadoop combinada com a velocidade e flexibilidade do processamento Spark.

A escolha entre Apache Hadoop e Apache Spark deve ser baseada nos requisitos específicos do projeto, considerando fatores como volume de dados, tipo de processamento necessário, recursos disponíveis, e expertise da equipe. Ambos os frameworks continuam evoluindo e mantêm relevância no ecossistema de big data, cada um atendendo a necessidades distintas do mercado.

VIII. CONCLUSÕES

Este trabalho apresentou uma análise abrangente do Apache Hadoop como middleware para sistemas distribuídos, explorando desde fundamentos teóricos até implementações práticas e testes experimentais. A investigação revelou que o Hadoop representa uma solução madura e robusta para os desafios do processamento distribuído de Big Data.

A. Principais Contribuições

O Apache Hadoop introduziu mudanças paradigmáticas significativas no processamento distribuído de dados. Sua principal contribuição reside na democratização das tecnologias de Big Data, tornando acessíveis para organizações de diversos portes capacidades anteriormente exclusivas de grandes corporações com recursos substanciais.

A arquitetura do Hadoop, baseada no paradigma MapReduce e no sistema de arquivos distribuído HDFS, oferece uma abstração elegante para processamento paralelo de grandes volumes de dados. Esta abstração permite que desenvolvedores foquem na lógica de negócio sem se preocupar com os complexos detalhes de implementação de sistemas distribuídos.

B. Limitações Identificadas

Apesar de suas qualidades, os testes e análises revelaram limitações importantes do Hadoop:

Latência de Processamento: O modelo MapReduce introduz latência significativa devido ao overhead de inicialização de jobs e operações de I/O em disco. Para aplicações que requerem processamento em tempo real ou baixa latência, o Hadoop tradicional pode não ser a solução ideal.

Complexidade de Configuração: Embora o Hadoop abstraia muitas complexidades de sistemas distribuídos, sua configuração e otimização ainda requerem conhecimento especializado.

Overhead para Datasets Pequenos: Para conjuntos de dados pequenos ou médios, o overhead do Hadoop pode superar seus benefícios. O sistema é otimizado para processamento de grandes volumes de dados.

C. Evolução e Tendências Futuras

O ecossistema Hadoop continua evoluindo para abordar suas limitações e adaptar-se às necessidades emergentes do Big Data. O desenvolvimento do YARN representou um marco importante, separando o gerenciamento de recursos do processamento de dados e permitindo a execução de diferentes frameworks sobre a mesma infraestrutura.

A integração com tecnologias de processamento de streaming, como Apache Storm e Apache Kafka, expande as capacidades do Hadoop para cenários de processamento em tempo real. Similarmente, a incorporação de tecnologias de computação em memória, exemplificada pelo Apache Spark, oferece alternativas de baixa latência ao MapReduce tradicional.

REFERÊNCIAS

- [1] A. S. Tanenbaum e M. Van Steen, *Sistemas Distribuídos: Princípios e Paradigmas*, 2ª ed. Prentice-Hall, 2007.
- [2] J. Dean e S. Ghemawat, "MapReduce: Processamento simplificado de dados em clusters grandes," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [3] D. Laney, "Gerenciamento de dados 3D: Controlando volume, velocidade e variedade de dados," *META Group Research Note*, vol. 6, no. 70, 2001.
- [4] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, e I. Stoica, "Spark: Computação em cluster com conjuntos de trabalho," *HotCloud*, vol. 10, no. 10-10, p. 95, 2010.
- [5] S. Ghemawat, H. Gobioff, e S. T. Leung, "O sistema de arquivos do Google," em *Proc. 19th ACM Symp. Operating Systems Principles*, 2003, pp. 29–43.
- [6] F. Chang et al., "Bigtable: Um sistema de armazenamento distribuído para dados estruturados," *ACM Trans. Computer Systems*, vol. 26, no. 2, pp. 1–26, 2008.
- [7] K. Shvachko, H. Kuang, S. Radia, e R. Chansler, "O sistema de arquivos distribuído hadoop," em *2010 IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.
- [8] V. K. Vavilapalli et al., "Apache hadoop yarn: Mais um negociador de recursos," em *Proc. 4th Annual Symp. Cloud Computing*, 2013, pp. 1–16.
- [9] T. White, *Hadoop: O Guia Definitivo*, 4ª ed. O'Reilly Media, 2012.
- [10] Apache Software Foundation, "Documentação do Apache Hadoop," 2023. [Online]. Disponível: <https://hadoop.apache.org/docs/>
- [11] E. A. Brewer, "Em direção a sistemas distribuídos robustos," em *Proc. 19th Annual ACM Symp. Principles of Distributed Computing*, 2000, pp. 7–10.
- [12] P. Hunt, M. Konar, F. P. Junqueira, e B. Reed, "ZooKeeper: Coordenação sem espera para sistemas em escala de internet," em *USENIX Annual Technical Conference*, vol. 8, 2010, pp. 9–9.
- [13] R. B. Standler, *Technical Writing*, Dover Publications, 2011.
- [14] Docker Inc., "Install Docker Engine on windows," *Docker Docs*, Jul. 2025. [Online]. Available: <https://docs.docker.com/desktop/setup/install/windows-install/>. [Accessed: Jul. 26, 2025].
- [15] IMB Inc., "O que é o Hadoop Distributed File System (HDFS)?" *Docker Docs*, Jul. 2025. [Online]. Available: <https://docs.docker.com/desktop/setup/install/windows-install/>. [Accessed: Jul. 26, 2025].
- [16] Apache Software Foundation. *Apache Hadoop 3.4.1 – MapReduce Tutorial*. Disponível em: <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>. Acesso em: jul. 2025.
- [17] VEENA, H.V.; CHITRA, K. A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. *Journal of Big Data*, v. 7, n. 110, 2020. Disponível em: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00388-5>. Acesso em: jul. 2025.
- [18] HASSAN, M.M.; HASSAN, M.M.; HUDA, S.; GHANI, I.; ISLAM, R.; ABAWAJY, J.H. MapReduce scheduling algorithms in Hadoop: a systematic study. *Journal of Cloud Computing*, v. 12, n. 136, 2023. Disponível em: <https://journalofcloudcomputing.springeropen.com/articles/10.1186/s13677-023-00520-9>. Acesso em: jul. 2025.
- [19] SANTOSO, L.W.; PRAMADHANI, K.N.; TJONDRONEGORO, S. Implementation of page rank algorithm in Hadoop MapReduce framework. In: *2016 International Conference on Informatics and Computing (ICIC)*, 2016. IEEE Xplore. Disponível em: <https://ieeexplore.ieee.org/document/7828663/>. Acesso em: jul. 2025.
- [20] SANTOSO, L.W.; PRAMADHANI, K.N.; TJONDRONEGORO, S. Page Rank Algorithm in Hadoop By MapReduce Framework. *ResearchGate*, nov. 2022. Disponível em: https://www.researchgate.net/publication/365834974_Page_Rank_Algorithm_in_Hadoop_By_MapReduce_Framework. Acesso em: jul. 2025.
- [21] TAYLOR, R.C. An overview of the Hadoop/MapReduce/HBase framework and its current applications in bioinformatics. *BMC Bioinformatics*, v. 11, n. S12, p. S1, 2010. Disponível em: <https://bmcbioinformatics.biomedcentral.com/articles/10.1186/1471-2105-11-S12-S1>. Acesso em: jul. 2025.
- [22] TAYLOR, R.C. An overview of the Hadoop/MapReduce/HBase framework and its current applications in bioinformatics. *PubMed*, PMID: 21210976, 2010. Disponível em: <https://pubmed.ncbi.nlm.nih.gov/21210976/>. Acesso em: jul. 2025.
- [23] KUMAR, A.; KUMAR, M. Hadoop and Its Role in Modern Image Processing. *ResearchGate*, out. 2014. Disponível em: https://www.researchgate.net/publication/267099174_Hadoop_and_Its_Role_in_Modern_Image_Processing. Acesso em: jul. 2025.
- [24] NOWICKI, M.; RYCZKOWSKA, M.; GÓRSKI, Ł. Comparison of sort algorithms in Hadoop and PCJ. *Journal of Big Data*, v. 7, n. 86, 2020. Disponível em: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00376-9>. Acesso em: jul. 2025.
- [25] CHOI, S.; LIM, H.; BAEK, H.; KIM, S.; CHANG, H. Task failure resilience technique for improving the performance of MapReduce in Hadoop. *ETRI Journal*, v. 42, n. 5, p. 672–684, 2020. Disponível em: <https://onlinelibrary.wiley.com/doi/10.4218/etrij.2018-0265>. Acesso em: jul. 2025.
- [26] KATAL, A.; WAZID, M.; GOUDAR, R.H. Big Data Processing with Hadoop-MapReduce in Cloud Systems. *ResearchGate*, nov. 2012. Disponível em: https://www.researchgate.net/publication/275405716_Big_Data_Processing_with_Hadoop-MapReduce_in_Cloud_Systems. Acesso em: jul. 2025.
- [27] Intel Corporation. *HiBench: Big Data Benchmark Suite*. GitHub Repository. Disponível em: <https://github.com/Intel-bigdata/HiBench>. Acesso em: jul. 2025.
- [28] VEENA, H.V.; CHITRA, K. A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. *Journal of Big Data*, v. 7, n. 110, 2020. Disponível em: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00388-5>. Acesso em: jul. 2025.
- [29] SINGH, P.; SINGH, N. Performance comparison of Apache Hadoop and Apache Spark. In: *Proceedings of the Third International Conference on Advanced Informatics for Computing Research*, 2019. ACM Digital Library. Disponível em: <https://dl.acm.org/doi/10.1145/3339311.3339329>. Acesso em: jul. 2025.
- [30] DataCamp. Hadoop vs Spark: Which Big Data Framework Is Right For

You? *DataCamp Blog*, abr. 2025. Disponível em: <https://www.datacamp.com/blog/hadoop-vs-spark>. Acesso em: jul. 2025.

- [31] Cloudlytics. Hadoop vs Spark: A Comparative Study. *Cloudlytics Blog*, jun. 2022. Disponível em: <https://cloudlytics.com/hadoop-vs-spark-a-comparative-study/>. Acesso em: jul. 2025.
- [32] Logz.io. Hadoop vs. Spark: A Head-To-Head Comparison. *Logz.io Blog*, set. 2023. Disponível em: <https://logz.io/blog/hadoop-vs-spark/>. Acesso em: jul. 2025.
- [33] Nebius. Spark vs. Hadoop in data engineering. *Nebius Blog*, set. 2024. Disponível em: <https://nebius.com/blog/posts/spark-vs-hadoop-in-data-engineering>. Acesso em: jul. 2025.
- [34] Amazon Web Services. Hadoop vs Spark - Difference Between Apache Frameworks. *AWS Documentation*, jul. 2025. Disponível em: <https://aws.amazon.com/compare/the-difference-between-hadoop-vs-spark/>. Acesso em: jul. 2025.
- [35] Coursera. Hadoop vs. Spark: What's the Difference? *Coursera Articles*, abr. 2024. Disponível em: <https://www.coursera.org/articles/hadoop-vs-spark>. Acesso em: jul. 2025.
- [36] PhoenixNAP. Hadoop vs Spark: Detailed Comparison of Big Data Frameworks. *PhoenixNAP Knowledge Base*, jan. 2025. Disponível em: <https://phoenixnap.com/kb/hadoop-vs-spark>. Acesso em: jul. 2025.
- [37] Integrate.io. Spark vs Hadoop MapReduce: 5 Key Differences. *Integrate.io Blog*, mar. 2023. Disponível em: <https://www.integrate.io/blog/apache-spark-vs-hadoop-mapreduce/>. Acesso em: jul. 2025.
- [38] GUPTA, P.; SHARMA, A.; JINDAL, R. A Critical Analysis of Apache Hadoop and Spark for Big Data Processing. In: *2021 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, 2021. IEEE Xplore. Disponível em: <https://ieeexplore.ieee.org/document/9609518/>. Acesso em: jul. 2025.
- [39] KHAN, M.; JIN, Y.; LI, M.; XIA, Y.; JIANG, C. Real-time data analysis using Spark and Hadoop. In: *2018 IEEE International Conference on Big Data and Smart Computing (BigComp)*, 2018. IEEE Xplore. Disponível em: <https://ieeexplore.ieee.org/document/8370593/>. Acesso em: jul. 2025.
- [40] RAO, A.R.; CLARKE, D. Performance Comparison of Apache Hadoop and Apache Spark for COVID-19 data sets. In: *2022 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, 2022. IEEE Xplore. Disponível em: <https://ieeexplore.ieee.org/document/9716232/>. Acesso em: jul. 2025.
- [41] SALLLOUM, S.; DAUTOV, R.; CHEN, X.; PENG, P.X.; HUANG, J.Z. Big data analytics on Apache Spark. *International Journal of Data Science and Analytics*, v. 1, n. 3-4, p. 145-164, out. 2016. Springer. Disponível em: <https://link.springer.com/article/10.1007/s41060-016-0027-9>. Acesso em: jul. 2025.
- [42] ZAHARIA, M.; XIN, R.S.; WENDELL, P.; DAS, T.; ARMBRUST, M.; DAVE, A.; MENG, X.; ROSEN, J.; VENKATARAMAN, S.; FRANKLIN, M.J.; GHODSI, A.; GONZALEZ, J.; SHENKER, S.; STOICA, I. Apache Spark: a unified engine for big data processing. *Communications of the ACM*, v. 59, n. 11, p. 56-65, nov. 2016. ACM Digital Library. Disponível em: <https://dl.acm.org/doi/10.1145/2934664>. Acesso em: jul. 2025.
- [43] BOURHIM, E.H.; CHERKAoui, A. Performance evaluation of Apache Hadoop and Apache Spark for parallelization of compute-intensive tasks. In: *Proceedings of the 22nd International Conference on Information Integration and Web-based Applications & Services*, 2020. ACM Digital Library. Disponível em: <https://dl.acm.org/doi/10.1145/3428757.3429121>. Acesso em: jul. 2025.
- [44] KUMAR, A.; GARG, N. Apache Spark: A Big Data Processing Engine. *ResearchGate*, nov. 2019. Disponível em: https://www.researchgate.net/publication/339176824_Apache_Spark_A_Big_Data_Processing_Engine. Acesso em: jul. 2025.
- [45] KATAL, A.; WAZID, M.; GOUDAR, R.H. Big Data Processing with Hadoop-MapReduce in Cloud Systems. *ResearchGate*, nov. 2012. Disponível em: https://www.researchgate.net/publication/275405716_Big_Data_Processing_with_Hadoop-MapReduce_in_Cloud_Systems. Acesso em: jul. 2025.
- [46] Integrate.io. Spark vs Hadoop MapReduce: 5 Key Differences. *Integrate.io Blog*, mar. 2023. Disponível em: <https://www.integrate.io/blog/apache-spark-vs-hadoop-mapreduce/>. Acesso em: jul. 2025.